

## MONITOR Z · L02

# Dormir sin gastar CPU: STIMER, WAIT, POST y ECB

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **\_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

# L02 — Dormir sin gastar CPU: STIMER, WAIT, POST y ECB

**Objetivo:** escribir el corazón de cualquier demonio, un **bucle que duerme** y despierta cuando corresponde, y **medir** que dormido no gasta CPU. Al final sabés combinar en una sola espera el timer, las órdenes del operador y los eventos.

## 1. El bucle

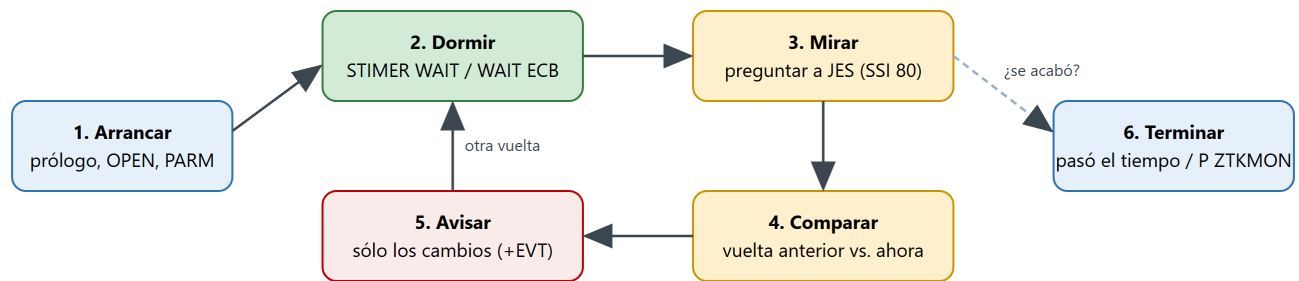
Todas las versiones del monitor tienen el mismo bucle de seis pasos:

1. **Arrancar:** prólogo, OPEN, leer el PARM (ya lo viste en L01).
2. **Dormir** hasta que pase el intervalo o alguien te despierte.
3. **Mirar:** preguntarle a JES (L03).
4. **Comparar** con la vuelta anterior.
5. **Avisar sólo los cambios** (+EVT INI / +EVT FIN).
6. **Terminar** cuando se cumple el tiempo o llega un P ZTKMON.

El paso 2 es el que hace que el demonio sea bueno o malo. En ZTKMONS v0 está en la línea 199:

|       |        |                    |                              |
|-------|--------|--------------------|------------------------------|
| NOLAT | DS     | 0H                 |                              |
|       | STIMER | WAIT,BINTVL=INTBIN | duermo el intervalo del PARM |
|       | L      | R1,ITER            | cuento la vuelta             |
|       | BCTR   | R1,0               |                              |
|       | ST     | R1,ITER            |                              |
|       | LTR    | R1,R1              |                              |
|       | BP     | BUCLE              | quedan vueltas → otra vez    |

Lo único que cambia entre el nivel B y el C (L00) es **quién despierta al bucle** en el paso 2: en B lo despierta el reloj, en C lo despierta JES.



## 🦊 Analogía: el guardia nocturno con despertador

Pone el despertador (2), se despierta y mira las cámaras (3), compara con lo que vio la última vez (4), anota sólo lo nuevo (5) y vuelve a poner el despertador. Casi toda la noche está en el paso 2: **durmiendo, sin gastar**.

Este bucle es el mismo en **todas** las versiones. Lo único que cambia entre el nivel B y el C es **quién lo despierta** en el paso 2: el reloj o JES.

Fuente: ZTKMONS.asm (bucle principal) · criterio propio para el diagrama

Lámina L02-01\_el\_bucle

## 2. STIMER: dormir un intervalo

| Forma                                     | Qué hace                                                                                                        | Unidad                                                      |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| STIMER WAIT, BINTVL=campo                 | La tarea <b>queda en espera</b> hasta que vence el intervalo                                                    | Palabra binaria en <b>centésimas de segundo</b> (500 = 5 s) |
| STIMER WAIT, DINTVL=campo                 | Igual, con el intervalo en decimal                                                                              | 8 bytes empaquetados HHMMSSth                               |
| STIMER REAL, rutina, BINTVL=...           | <b>No espera</b> : el programa sigue y, al vencer, el sistema llama a la <b>rutina de salida</b>                | Igual                                                       |
| STIMERM SET, ID=..., BINTVL=..., EXIT=... | Como REAL, pero con un <b>identificador</b> : se puede tener más de uno y cancelarlo con STIMERM CANCEL, ID=... | Igual                                                       |

En ZTKMONS, `INTBIN` se calcula a partir del PARM: segundos × 100.

**El límite de STIMER WAIT**: mientras duerme, **no puede atender nada más**. Si el operador manda `P ZTKMON` en el segundo 1 de un intervalo de 60 s, el

programa se entera recién en el segundo 60. Para un job de prueba alcanza; para el STC no. Por eso existe la sección 4.

 Fuente: *z/OS MVS Programming: Assembler Services Reference* (STIMER, STIMERM) · *Assembler Services Guide*, capítulo de servicios de tiempo · `ZTKMONS.asm` línea 199.

STIMER: dormir un intervalo

L02 · Dormir sin gastar CPU

| Forma                         | Qué hace                                                                                        | Unidad del intervalo                                        |
|-------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| STIMER WAIT,BINTVL=campo      | La tarea <b>queda en espera</b> hasta que vence el intervalo                                    | Palabra binaria en <b>centésimas de segundo</b> (500 = 5 s) |
| STIMER WAIT,DINTVL=campo      | Igual, con el intervalo en decimal                                                              | 8 bytes empaquetados HHMMSSth                               |
| STIMER REAL,rutina,BINTVL=... | <b>No</b> espera: sigue trabajando y, al vencer, el sistema llama a una <b>rutina de salida</b> | Igual que arriba                                            |
| STIMERM SET,ID=...,EXIT=...   | Como REAL pero con <b>varios timers</b> identificados; se cancela con STIMERM CANCEL            | BINTVL / DINTVL / ...                                       |

```
SEG5    DC    F'500'           5 segundos = 500 centésimas
...
VUELTA  DS    0H
        STIMER WAIT,BINTVL=SEG5 duermo 5 s (CPU = 0)
        BAL   R14,MIRAR        pregunto a JES
        B     VUELTA
```

 **Analogía: el temporizador de cocina**

STIMER WAIT es poner el temporizador y **sentarse a esperar** que suene. STIMER REAL es ponerlo y **seguir cocinando**: cuando suena, alguien (la rutina de salida) te toca el hombro.

Con STIMER WAIT el bucle **no puede atender otra cosa** mientras duerme (por ejemplo, un P ZTKMON). Para eso se combina timer + ECB (siguiente lámina).

Fuente: *z/OS MVS Programming: Assembler Services Reference* (STIMER, STIMERM) · *Assembler Services Guide* (timing services)

Lámina L02-02\_stimer

### 3. El ECB: un semáforo de una palabra

Un **ECB** (*event control block*) es una palabra (4 bytes) alineada a palabra:

| Bits                                   | Significado                                             |
|----------------------------------------|---------------------------------------------------------|
| <b>bit 0</b> (X'80' en el primer byte) | "Alguien está esperando": lo pone WAIT                  |
| <b>bit 1</b> (X'40')                   | "Completo": lo pone POST, y eso despierta al que espera |
| bits 2–31                              | <b>Código de finalización</b> : lo que mandó el POST    |

| Instrucción             | Qué hace                                                | En Unix               |
|-------------------------|---------------------------------------------------------|-----------------------|
| XC ECB,ECB              | Limpia el ECB <b>antes</b> de esperar                   | reiniciar el semáforo |
| WAIT ECB=ECB            | Duerme hasta que el ECB esté completo                   | sem_wait()            |
| POST ECB,código         | Lo marca completo y despierta al que espera             | sem_post()            |
| WAIT<br>1,ECBLIST=LISTA | Duerme hasta que <b>uno</b> de varios ECB esté completo | select()              |

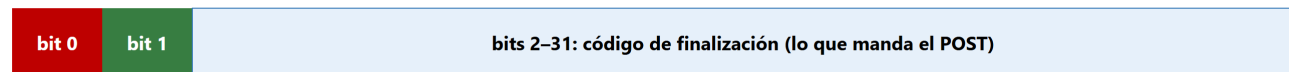
**Trampa clásica:** si no limpiás el ECB después de atenderlo, sigue con el bit "completo" encendido y el próximo WAIT **pasa de largo**. El bucle gira sin dormir y vuelve a ser un mal demonio.

 Fuente: *Assembler Services Guide*, "Synchronizing tasks" (WAIT, POST y el formato del ECB).

## El ECB: un semáforo de una palabra

L02 · Dormir sin gastar CPU

ECB = 1 palabra (4 bytes, 32 bits), alineada a palabra



X'80' = "alguien espera" (lo pone WAIT)

X'40' = "completo" (lo pone POST): el que espera se despierta

| Instrucción          | Qué hace                                                                         | Equivalente Unix      |
|----------------------|----------------------------------------------------------------------------------|-----------------------|
| XC ECB,ECB           | Limpia el ECB <b>antes</b> de esperar (si quedó el X'40', el WAIT pasa de largo) | reiniciar el semáforo |
| WAIT ECB=ECB         | La tarea duerme hasta que el ECB esté completo                                   | sem_wait()            |
| POST ECB,código      | Marca completo (X'40') + código, y despierta al que espera                       | sem_post()            |
| WAIT 1,ECBLIST=LISTA | Duerme hasta que <b>uno</b> de varios ECB esté completo                          | select()              |

### Analogía: la banderita del buzón

El ECB es la **banderita** de un buzón de campo. Bajás la banderita (XC) y te vas a dormir (WAIT). Cuando el cartero deja algo, **levanta la banderita** (POST) y eso te despierta. El "código" es la nota que dejó: qué tipo de correspondencia trajo.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Synchronizing tasks": WAIT, POST y formato del ECB)

Lámina L02-03\_ecb

## 4. Timer + operador + eventos en una sola espera

La forma de que el bucle atienda todo sin demora es esperar en **una lista de ECB**:

| ECB                        | Quién lo postea                    | Cuándo                         |
|----------------------------|------------------------------------|--------------------------------|
| ECBTIEMPO                  | La rutina de salida de STIMERM SET | Al vencer el intervalo         |
| El del COM (COMECBPT, L07) | El sistema                         | Cuando el operador manda F o P |
| ECBEVENTO                  | El listen exit de ENF 70 (L08)     | Cuando un job cambia de estado |

|       |      |                 |                                                  |
|-------|------|-----------------|--------------------------------------------------|
| LISTA | DC   | A(ECBTIEMPO)    | 1. <sup>a</sup> : dirección del ECB del timer    |
|       | DC   | A(0)            | 2. <sup>a</sup> : dirección del ECB del COMM (se |
|       | DC   | A(ECBEVENTO)    | 3. <sup>a</sup> y ÚLTIMA: dirección del ECB de e |
| ...   |      |                 |                                                  |
|       | OI   | LISTA+8,X'80'   | bit alto en la ÚLTIMA entrada = fin              |
|       | WAIT | 1,ECBLIST=LISTA | duermo hasta que 1 de los 3 esté cor             |

Al despertar se mira **cuál** se posteó (bit X'40' de cada ECB), se atiende sólo eso, se limpia **ese** ECB y se vuelve a esperar.

**La rutina de salida del timer** corre de forma **asíncrona**, en la misma tarea, mientras el programa principal está en WAIT. Tiene que ser **corta**: hacer POST ECBTIEMPO y volver. La forma exacta de su enlace (qué registros recibe y cómo vuelve) se escribe y se prueba en la práctica.

 Fuente: *Assembler Services Guide* (WAIT con ECBLIST; STIMERM con rutina de salida).  
Diseño de la lista: criterio propio.



|       |      |                 |                                                             |
|-------|------|-----------------|-------------------------------------------------------------|
| LISTA | DC   | A(ECBTIEMPO)    | 1.ª: dirección del ECB del timer                            |
|       | DC   | A(0)            | 2.ª: dirección del ECB del COMM (COMECBPT, se llena en L07) |
|       | DC   | A(ECBEVENTO)    | 3.ª y ÚLTIMA: dirección del ECB de eventos                  |
| ...   |      |                 |                                                             |
|       | OI   | LISTA+8,X'80'   | bit alto encendido en la ÚLTIMA entrada = fin de la lista   |
|       | WAIT | 1,ECBLIST=LISTA | duermo hasta que 1 de los 3 esté completo                   |

## 🔔 Analogía: la recepción de un hotel con tres timbres

El recepcionista duerme en su silla. Hay tres timbres: el reloj despertador, el teléfono del gerente y el timbre de la puerta. Se despierta con **el primero que suena**, mira cuál fue y atiende sólo eso.

La forma exacta de armar la lista y la rutina de salida del timer se escribe y prueba en la práctica de L02.

Fuente: Assembler Services Guide (WAIT con ECBLIST, STIMERM con rutina de salida) - criterio propio para el diseño

Lámina L02-04\_timer\_y\_eventos\_juntos

## 5. Cómo se mide que "dormido no gasta"

Al terminar un paso, el sistema escribe su consumo en el JESYSMSG y en el resumen del JESMSGGLG:

- IEF032I STEP/... /STOP ... CPU: 0 HR 00 MIN 00.01 SEC → CPU del paso.
- La columna **CPU** del resumen por paso del JESMSGGLG (la misma que lee la bitácora).

La cuenta es **CPU total ÷ cantidad de vueltas = costo de despertar una vez**. Ese número es la base para medir después lo que agrega SSI 80 (L04) y para decidir el intervalo en producción (L11).

## 6. Preguntas de comprobación

1. ¿Qué pasa con la CPU si te olvidás el `XC ECB,ECB` después de atender un `POST`? ¿Por qué?
2. ¿Cuántas centésimas pondrías en `BINTVL` para dormir 2,5 segundos?

- ## Mis respuestas

Partí de tu ZTKHOLA (P01):

Traé:

- el JOBID de cada corrida;
- la CPU del paso ( IEF032I );
- la diferencia de hora entre dos líneas seguidas del SYSPRINT.

El número que dé **no lo sabemos todavía**: es justamente lo que mide la práctica.

**Extra (opcional):** reemplazá STIMER WAIT por STIMERM SET + WAIT sobre un ECB que postea la rutina de salida. Es el primer paso hacia la ECBLIST del STC.

Práctica P02: el bucle y su CPU

L02 · Dormir sin gastar CPU

| Paso | Qué hace el programa ZTKBUCLE                                           | Qué aprendés        |
|------|-------------------------------------------------------------------------|---------------------|
| 1    | PARM 'segundos minutos' , por ejemplo '5 2'                             | Reusar PARSEA (L01) |
| 2    | En cada vuelta: STIMER WAIT y escribir la hora ( TIME DEC ) en SYSPRINT | El bucle y el reloj |
| 3    | Termina a los M minutos, con la cantidad de vueltas                     | Cortar el bucle     |
| 4    | Correr con '5 2' (24 vueltas) y con '1 2' (120 vueltas)                 | Comparar la CPU     |

| Qué mirar en el JESMSG LG / JESYSMSG                          | Qué esperamos                                                                     |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------|
| CPU del paso ( IEF032I ... CPU: o la columna CPU del resumen) | Muy baja: centésimas de segundo, aunque el job dure 2 minutos                     |
| Hora entre líneas del SYSPRINT                                | Muy cerca del intervalo pedido                                                    |
| CPU ÷ vueltas                                                 | El costo de "despertar y escribir". Es la base para medir después el SSI 80 (L04) |

El número que salga **no lo sabemos todavía**: es justamente lo que mide la práctica.

Fuente: Ruta de aprendizaje, módulo 2

Lámina L02-05\_practica\_medir\_cpu

Relacionado: [L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS](#) · [Glosario](#) · siguiente: [L03 - Hablarle a JES por SSI 80](#).

 Fuente: vivo\host\ZTKMONS.asm (bucle, líneas 190–206; AHORA, línea 234) y los manuales de IBM citados en cada sección.

